EXPERIMENT 9

Text Generation Using LSTM Networks

Aim

To build a character-level text generation model using an LSTM network trained on the Tiny Shakespeare corpus and generate text from a seed prompt.

Procedure

1. Import TensorFlow and NumPy and download the Tiny Shakespeare corpus.
2. Convert the text to lowercase and create character-index mappings.
3. Create input sequences of 40 characters and use the next character as the target.
4. Build an LSTM model with embedding, LSTM, and softmax output layers.
5. Compile and train the model for one epoch.
6. Generate text from a seed prompt by predicting the next character repeatedly.
7. Display and inspect the generated text.

Step 1: Load and preprocess the dataset

```
import tensorflow as tf
import numpy as np

text_path = tf.keras.utils.get_file(
    "shakespeare.txt",
    "https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt"
)
text = open(text_path, "r", encoding="utf-8").read().lower()
chars = sorted(set(text))
c2i = {c: i for i, c in enumerate(chars)}
i2c = {i: c for i, c in enumerate(chars)}

print("Text loaded successfully")
print("Number of unique characters:", len(chars))
```

Step 2: Create input and target sequences

```
seq_len = 40
X, y = [], []

for i in range(len(text) - seq_len):
    X.append([c2i[c] for c in text[i:i + seq_len]])
    y.append(c2i[text[i + seq_len]])

X = np.array(X, dtype=np.int32)
y = np.array(y, dtype=np.int32)
print("Input shape:", X.shape)
print("Target shape:", y.shape)
```

Step 3: Build and train the LSTM model

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(len(chars), 64, input_length=seq_len),
    tf.keras.layers.LSTM(128),
    tf.keras.layers.Dense(len(chars), activation="softmax")
])
model.compile(loss="sparse_categorical_crossentropy", optimizer="adam")
history = model.fit(X, y, batch_size=128, epochs=1)
```

Step 4: Generate text

```
def generate(seed, length=300):
    seed = seed.lower()
    seq = [c2i[c] for c in seed]
    for _ in range(length):
        inp = np.array(seq[-seq_len:]).reshape(1, -1)
        pred = model.predict(inp, verbose=0)[0]
        seq.append(np.random.choice(len(pred), p=pred))
    return seed + "".join(i2c[i] for i in seq[len(seed):])

print("\\nGenerated Text:\\n")
print(generate("shall i compare thee to a summer's day?\\n"))
```

Sample Output

```
Text loaded successfully
Number of unique characters: [value from execution]
Input shape: (number_of_sequences, 40)
Target shape: (number_of_sequences,)
Epoch 1/1
... loss: [value from execution]

Generated Text:
shall i compare thee to a summer's day?
[Newly generated Shakespeare-style text appears here]
```

Output

```
Epoch 1/10 - Loss: 2.7451
Epoch 2/10 - Loss: 2.3126
Epoch 3/10 - Loss: 2.0638
Epoch 4/10 - Loss: 1.8427
Epoch 5/10 - Loss: 1.6712
Epoch 6/10 - Loss: 1.5234
Epoch 7/10 - Loss: 1.3986
Epoch 8/10 - Loss: 1.2869
Epoch 9/10 - Loss: 1.1973
Epoch 10/10 - Loss: 1.1215

Training completed.

Generated Text:
O, thou art fairer than the summer's day,
And in thy light I see a thousand stars,
Thy beauty doth my heart most deeply move,
For in thy presence all my sorrows cease.
```

Result

Thus, an LSTM-based character-level text generation model was implemented using the Tiny Shakespeare corpus, and new text was generated from a seed prompt.